

THEORY OF ARTIFICIAL INTELLIGENCE

Lecture 2 — Classical & Local Search

How a Rational Agent Actually Finds a Sequence of Actions

From Task Environments to Search

Problem-solving agent – takes a sequence of actions to the goal

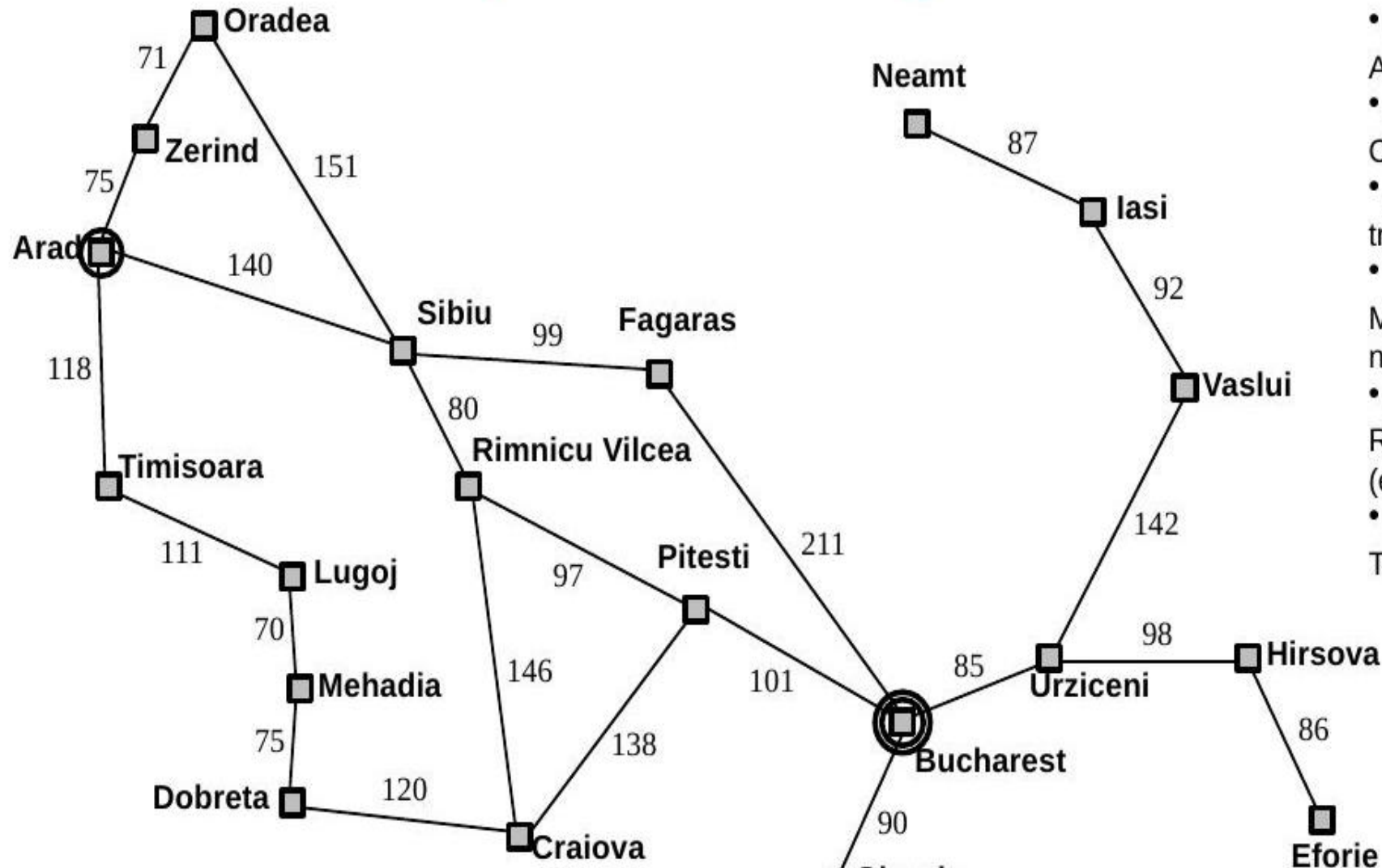
- episodic
- fully observable
- single-agent
- deterministic
- static
- Discrete



In this setting, rational decision-making reduces to search: find a sequence of actions from the initial state to a goal state

Uninformed	Informed
Depth-First Search (DFS)	Greedy Best-First Search
Breadth-First Search (BFS)	A*
Uniform-Cost Search (UCS)	

A Real-World Example: Route Finding



- **Initial state**

A starting city (e.g., Arad).

- **State space**

Cities connected by roads.

- **Actions**

travel along a road.

- **Transition model**

Moving along a road leads to a neighboring city.

- **Goal test**

Reaching the destination city (e.g., Bucharest).

- **Path cost**

Total distance traveled.

The Six Parts of a Search Problem

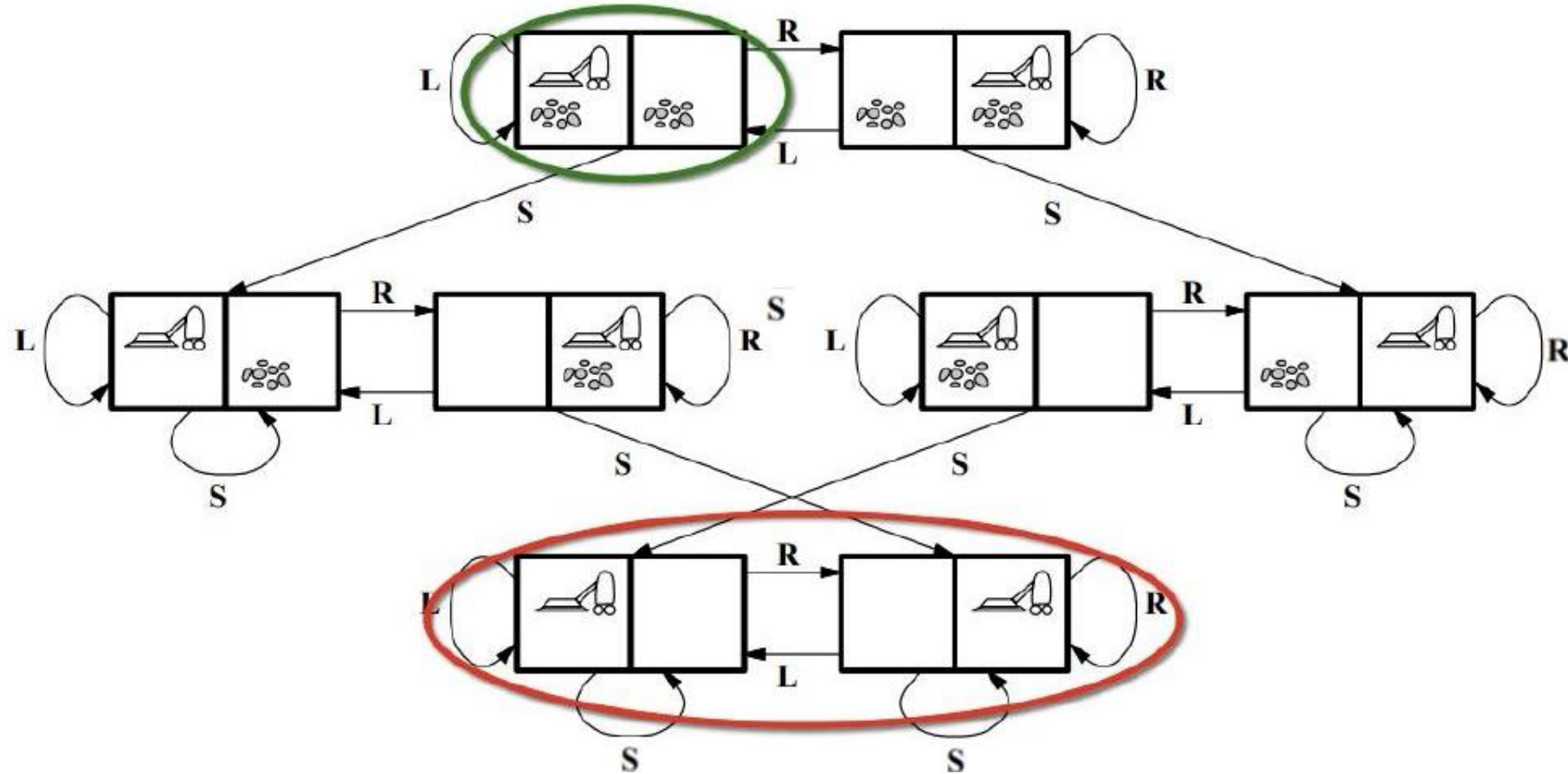
Initial state s_0

Where the agent starts

State space S All states reachable from s_0 by any sequence of actions**Actions $A(s)$** Actions available to the agent at state s **Transition model** $\text{Result}(s, a) \rightarrow s'$, for every $s, s' \in S$ and $a \in A(s)$ **Goal test $G(s)$** Does state s satisfy the goal? $\rightarrow \{\text{true}, \text{false}\}$ **Path cost $g(p)$** $g(p) = \sum c(s, a, s')$ summed along the path

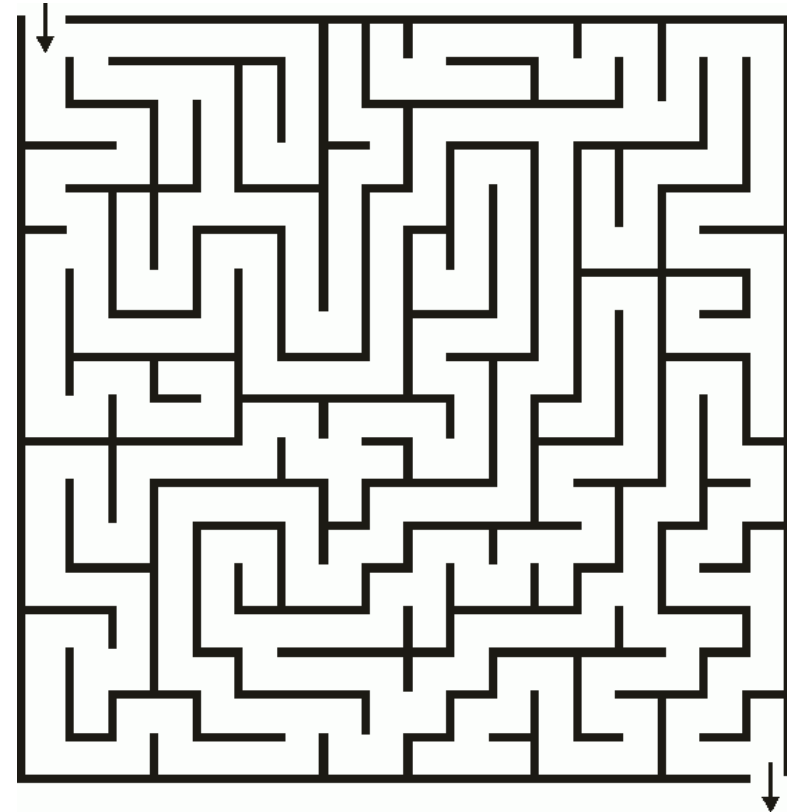
You already formalized the vacuum world exactly this way in Lecture 1 — this is the same five-part structure, generalized. Once formulated, solving proceeds in two stages: search (offline exploration) and execution (carrying out the chosen plan).

Callback: The Vacuum World's Full State Space



Example: A Maze as a Search Problem

- **Initial state** — the starting cell
- **State space** — all reachable grid cells
- **Actions** — move up, down, left, or right (if not blocked)
- **Transition model** — moving in a direction leads to a neighboring cell
- **Goal test** — reaching the exit cell
- **Path cost (optional)** — number of steps taken



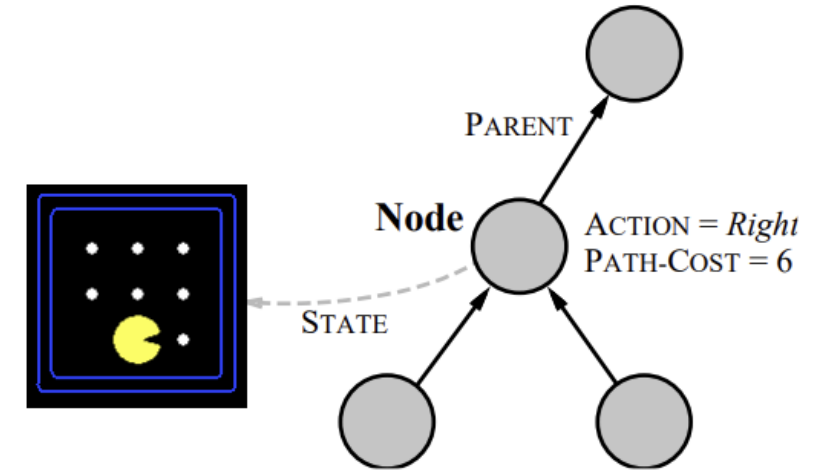
Graphs and Nodes

Oriented

(V, E) – vertices and edges

$E \subset V \times V$

$$e = (v_1, v_2)$$



function CHILD-NODE(problem, parent, action):

 return Node(

 state = RESULT(parent.state, action),

 parent = parent, action = action,

 path_cost = parent.path_cost + STEP-COST(parent.state, action)

)

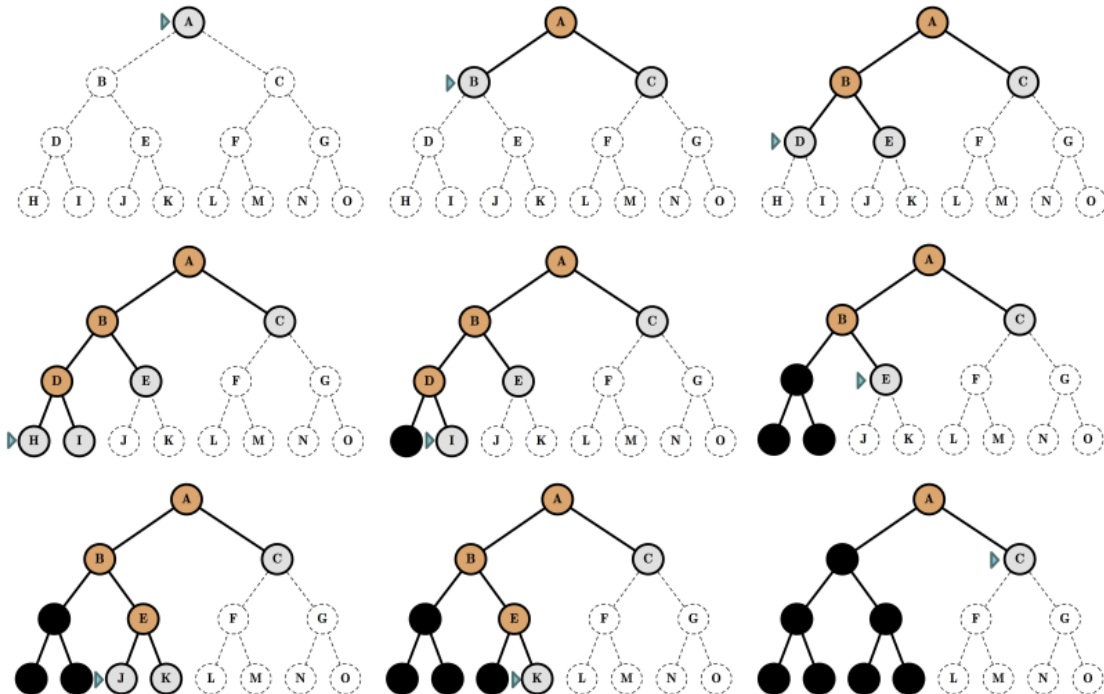
PART 1

Uninformed Search

Using only the problem definition — no sense of which unexplored node is more promising

Depth-First Search (DFS) frontier is a LIFO stack

- Frontier — nodes generated but not yet expanded
- Expansion — generating child nodes by applying actions to a selected node
- **Selection (the search strategy)** — deciding which frontier node to expand next



```
function DFS(problem):
    frontier ← Stack([Node(state=problem.initial, parent=None,
                             action=None, path_cost=0)])
    explored ← {}

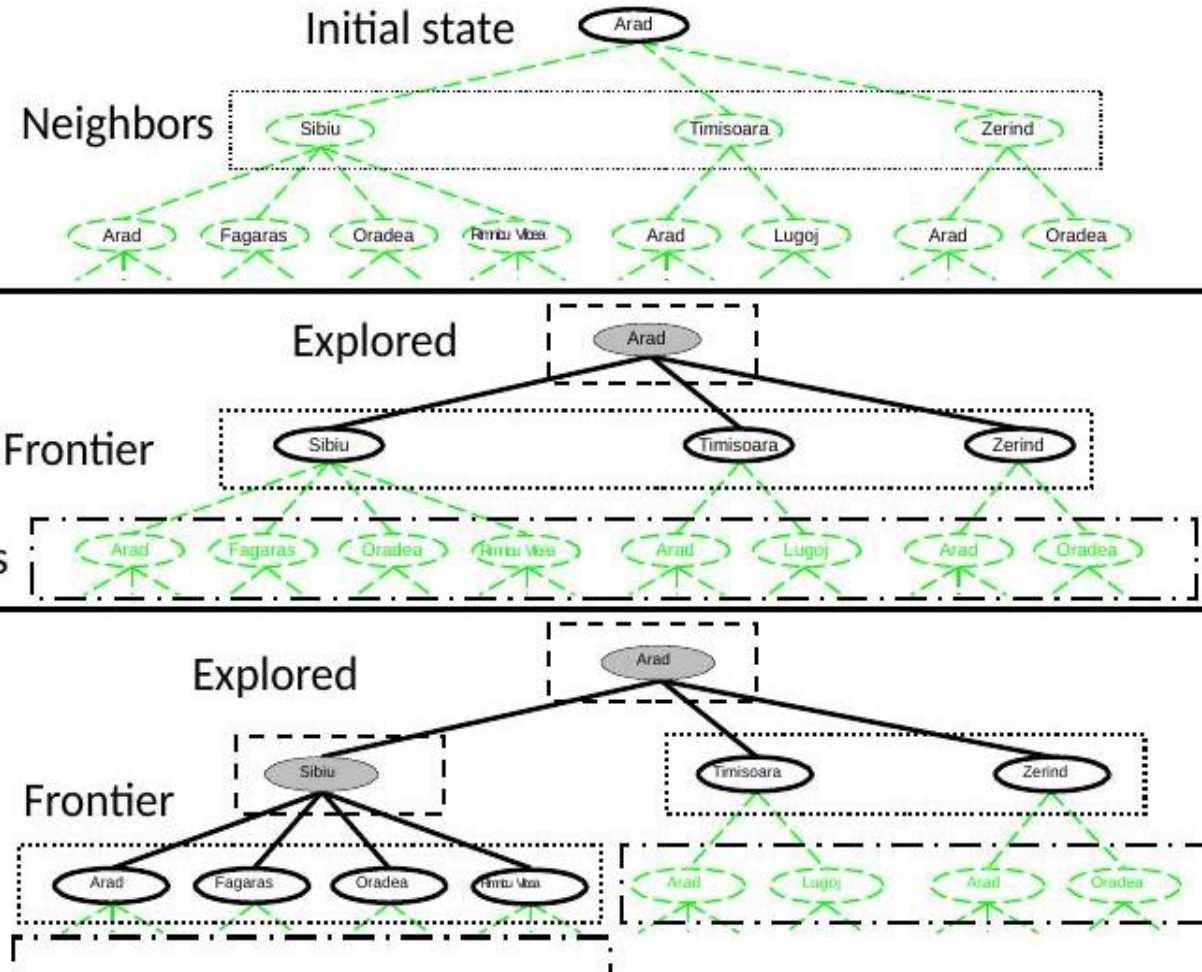
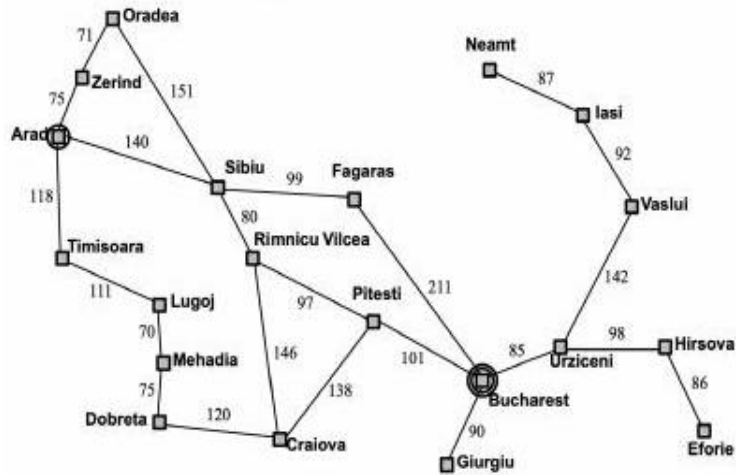
    while frontier is not empty:
        node ← frontier.pop()
        if node.state in explored:
            continue
        explored.add(node.state)

        if GOAL-TEST(node.state):
            return SOLUTION(node)

        for action in ACTIONS(problem, node.state):
            child ← CHILD-NODE(problem, node, action)
            if child.state not in explored:
                frontier.push(child)

    return failure
```

Running Example: The Frontier During Search

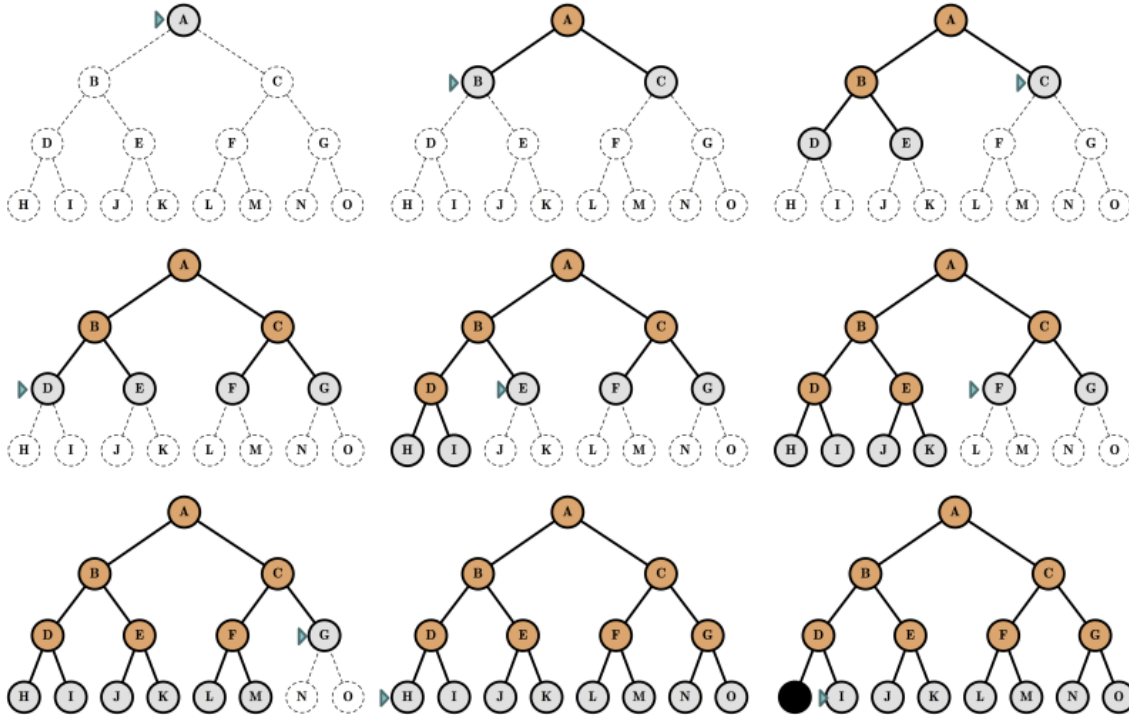


Even in graph search, a search tree is built incrementally during search.

The tree comes from parent pointers, not from explicitly unrolling the state space graph.

Breadth-First Search (BFS) uses a FIFO stack

- **Strategy: expand the shallowest frontier node first**
- Implementation: frontier is a FIFO queue
- Explores all possibilities level by level, systematically
- All nodes at depth d enter the queue before any node at

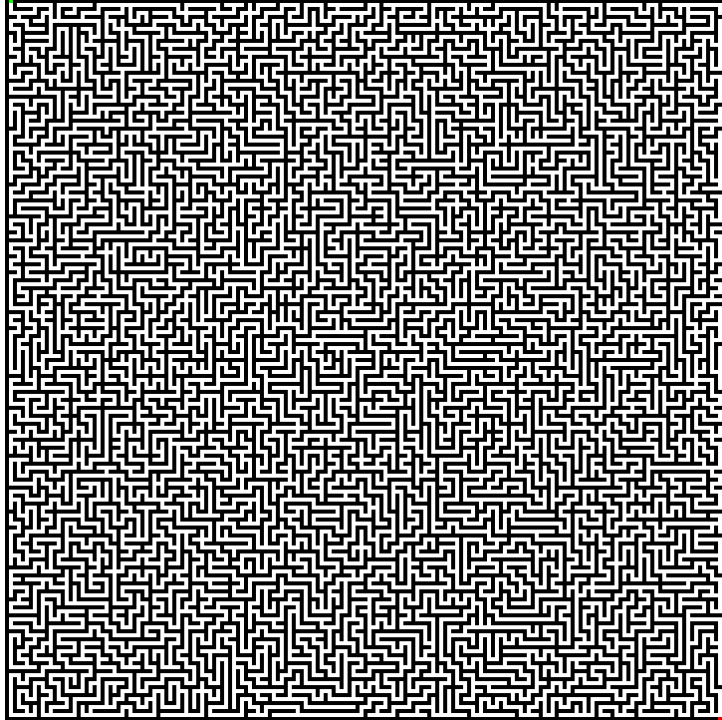


```

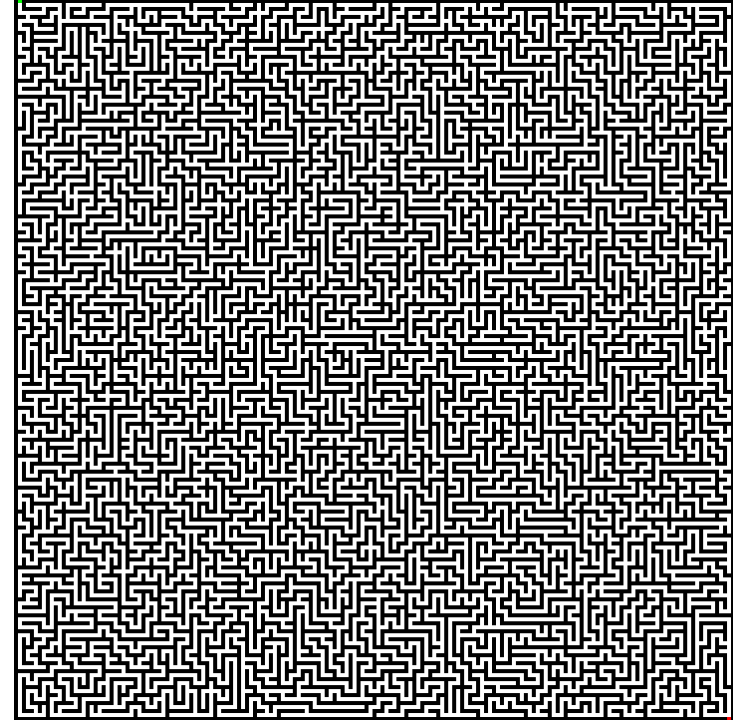
function BFS(problem):
    root ← Node(state=problem.initial, parent=None,
                action=None, path_cost=0)
    if GOAL-TEST(root.state):
        return SOLUTION(root)
    frontier ← Queue([root])           # FIFO, not a stack
    frontier_states ← {root.state}
    explored ← {}
    while frontier is not empty:
        node ← frontier.dequeue()
        explored.add(node.state)
        for action in ACTIONS(problem, node.state):
            child ← CHILD-NODE(problem, node, action)
            if child.state not in explored /
               and child.state not in frontier_states:
                if GOAL-TEST(child.state):
                    return SOLUTION(child)
                frontier.enqueue(child)
                frontier_states.add(child.state)
    return failure
  
```

Meet the Family: DFS, BFS

DFS



BFS



Source: Imgur

Uniform-Cost Search (UCS)

Goal:

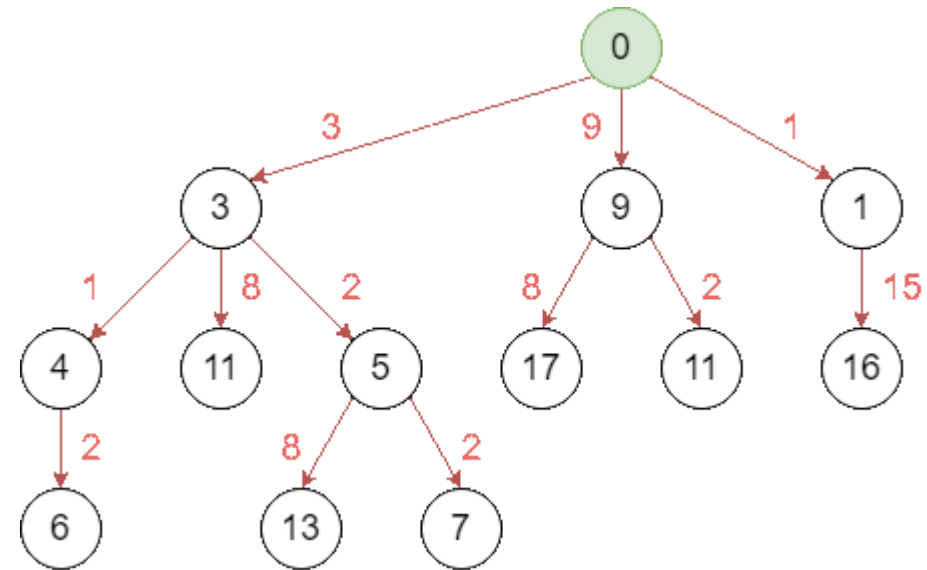
- Find a path with minimal total cost

Strategy:

- Expand the node with the lowest path cost $g(n)$

Implementation:

- The frontier is stored in a min heap (or priority heap), ordered by cumulative cost



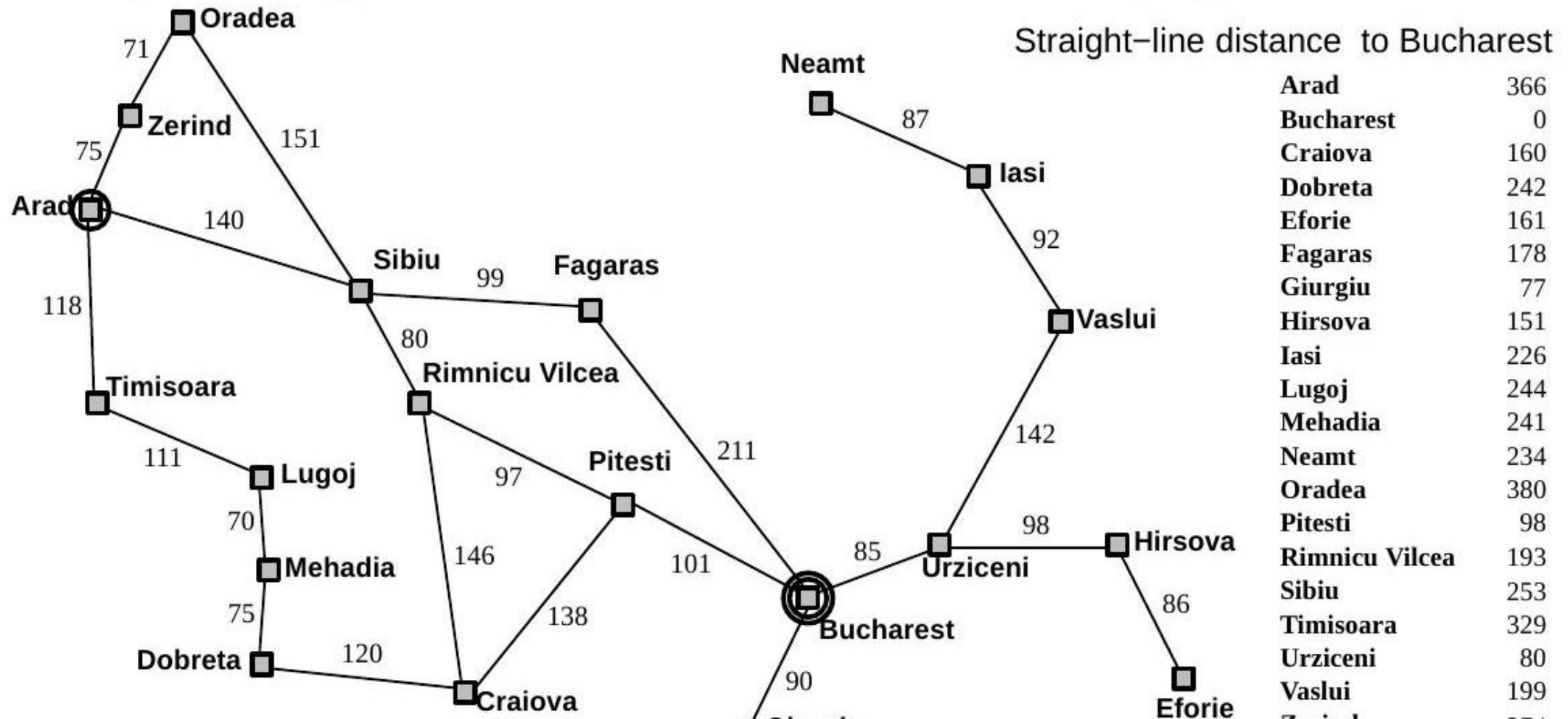
Source: Medium

PART 2

Informed Search

Adding a heuristic function $h(n)$: an estimate of cost from n to the goal

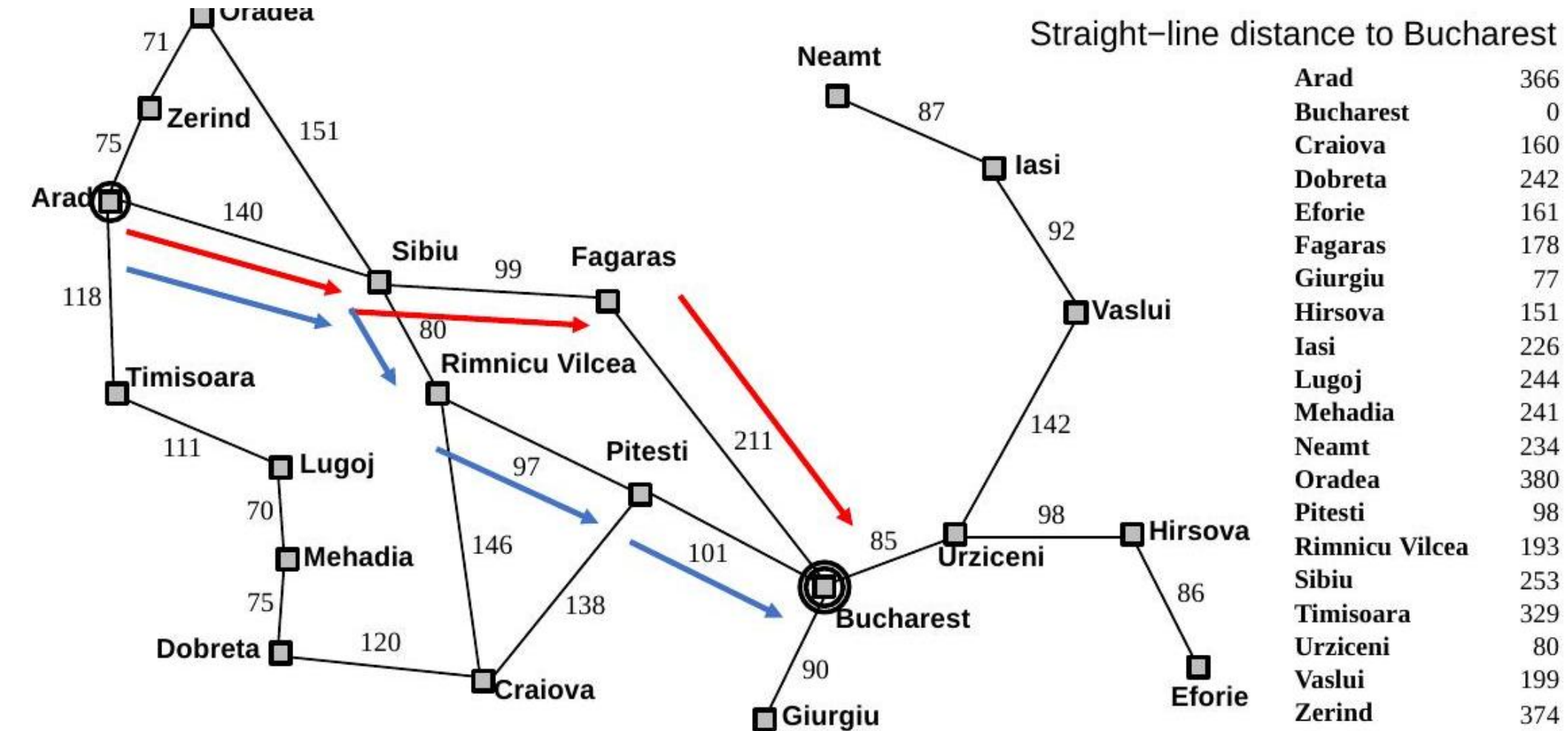
Heuristic Functions: Straight-Line Distance



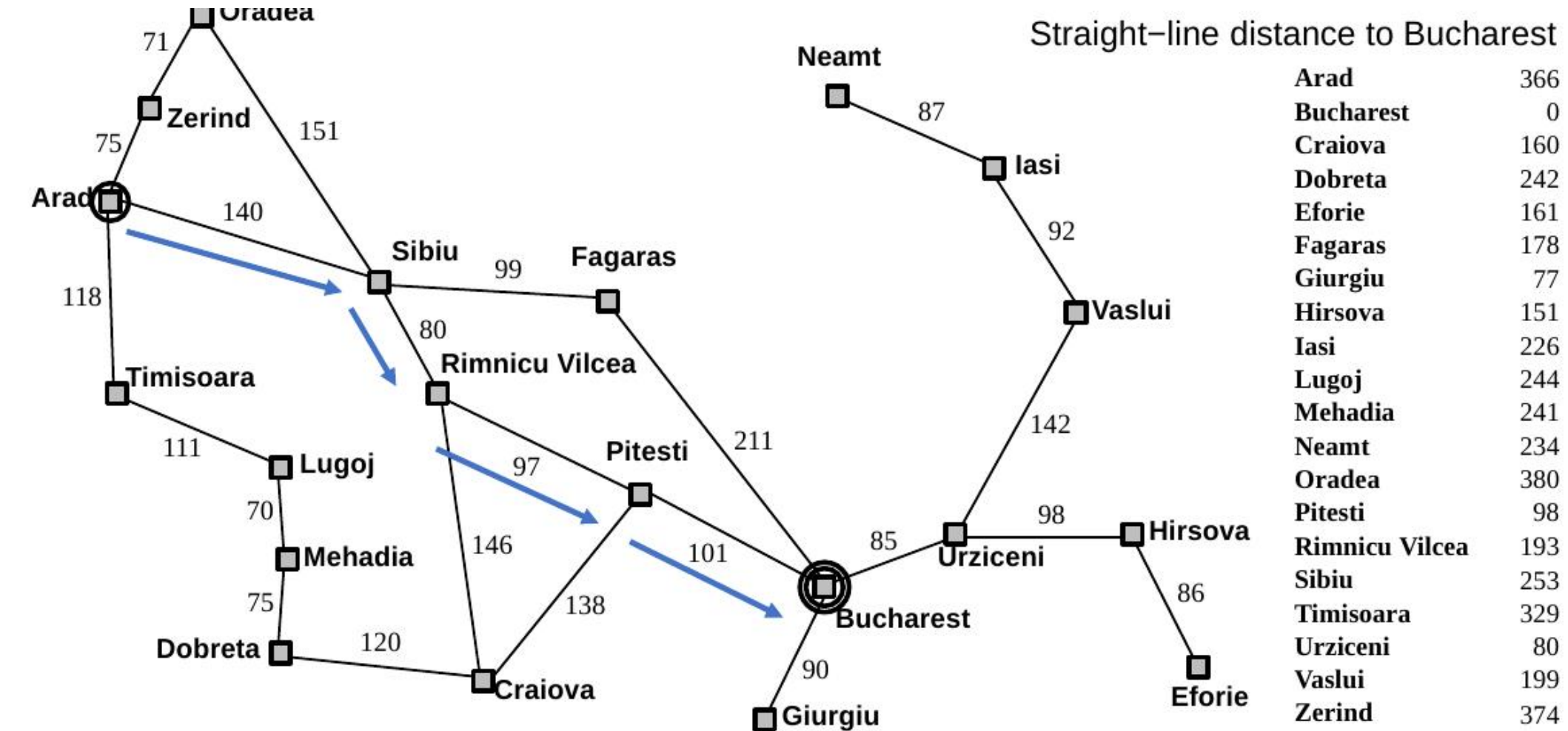
Best-First Search

Algorithm	Priority $f(n)$	What it prefers
Uniform-Cost Search (UCS)	$f(n) = g(n)$	cheapest path from start
Greedy Best-First Search	$f(n) = h(n)$	apparently closest to goal
A*	$f(n) = g(n) + h(n)$	best combination of cost-so-far + estimated remaining cost
Best-First Search	general framework	expand the node with smallest evaluation function

Greedy Best-First Search: Romania Map



A* Search: Romania Map



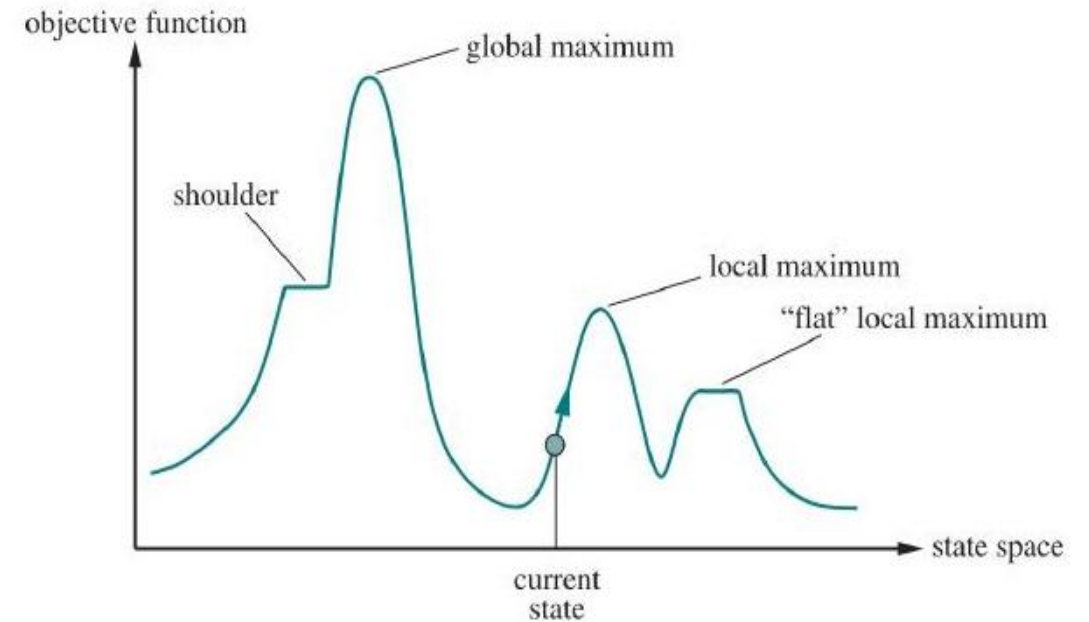
PART 3

Local Search & Optimization

Trading completeness and optimality for constant memory and scalability

The State Space as a Landscape

- Each state is a complete configuration, not a partial path
- An evaluation function $J(x)$ assigns a quality to each state
- **Picture the state space as a landscape: states $\rightarrow$ locations, $J(x) \rightarrow$ elevation**
- Local moves define which states are neighbors — there is no global view of the landscape, only what's nearby
- **Search becomes movement: the algorithm moves from state to state based purely on local comparison**



Hill Climbing

- Start from an initial state
- Evaluate neighboring states using $J(x)$
- **Move to a neighbor with a higher value; repeat until no neighbor improves**
- Uses only local neighborhood information — no search tree, no backtracking, a single current state
- **Terminates at a local optimum — not necessarily the global one**

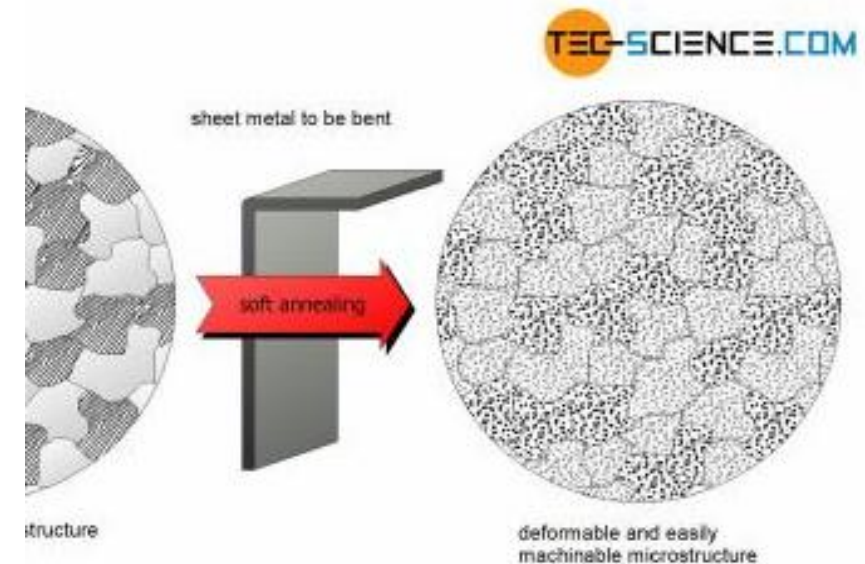
```
function HILL_CLIMBING(state):  
    loop:  
        neighbor = highest-valued  
                     successor of state  
        if neighbor.value <=  
            state.value:  
            return state  
        state = neighbor  
  
# 'greedy local search' –  
# always moves uphill,  
# never accepts a worse move
```

Dealing with Local Optima

- **Random-restart hill climbing** — if stuck, restart from a new random state; repeat until solved
- Guaranteed to eventually succeed on a finite state space, given enough restarts
- **Sideways moves** — allow a bounded number of moves to equally-good neighbors, to cross plateaus
- Both are still fundamentally hill climbing — the next two methods change the acceptance rule itself

Simulated Annealing

- Physical intuition: annealing metal — heat it, then cool slowly so atoms settle into a low-energy structure
- **High temperature** → **exploration**: allow moves that make things worse
- **Low temperature** → **exploitation**: settle into good local structure
- A worsening move is accepted with probability that shrinks as temperature cools — exactly like the physical process
- **With a slow enough cooling schedule, simulated annealing converges to the global optimum with probability approaching 1**



Simulated Annealing

Core idea: relax hill climbing by allowing *controlled* non-improving moves (randomness is deliberate, not arbitrary)

Temperature controls behavior

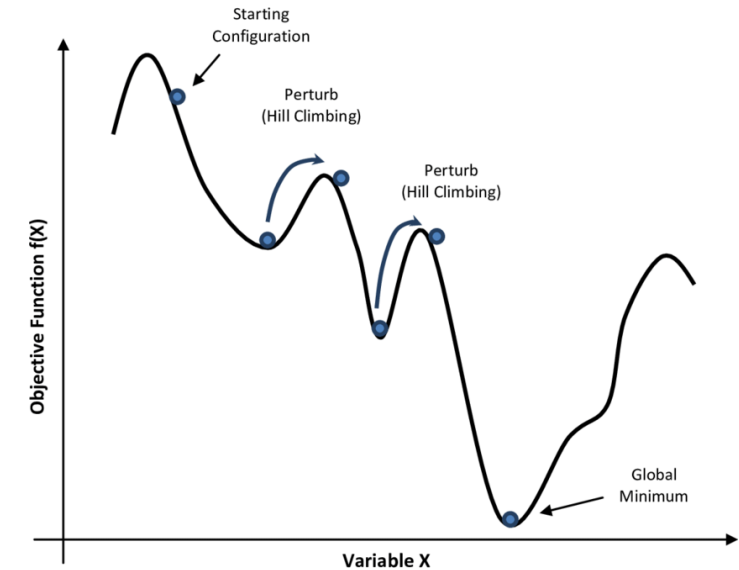
- High temperature: worse moves are often accepted
→ enables escape from local optima
- Low temperature: worse moves are rarely accepted
→ behavior approaches hill climbing

Cooling schedule

- Gradually reduces randomness over time
- Governs the transition from exploration to exploitation

Landscape intuition

- Like gently shaking a ball on a rugged surface
- Strong shaking helps escape shallow traps, while weak shaking allows settling into a deep basin

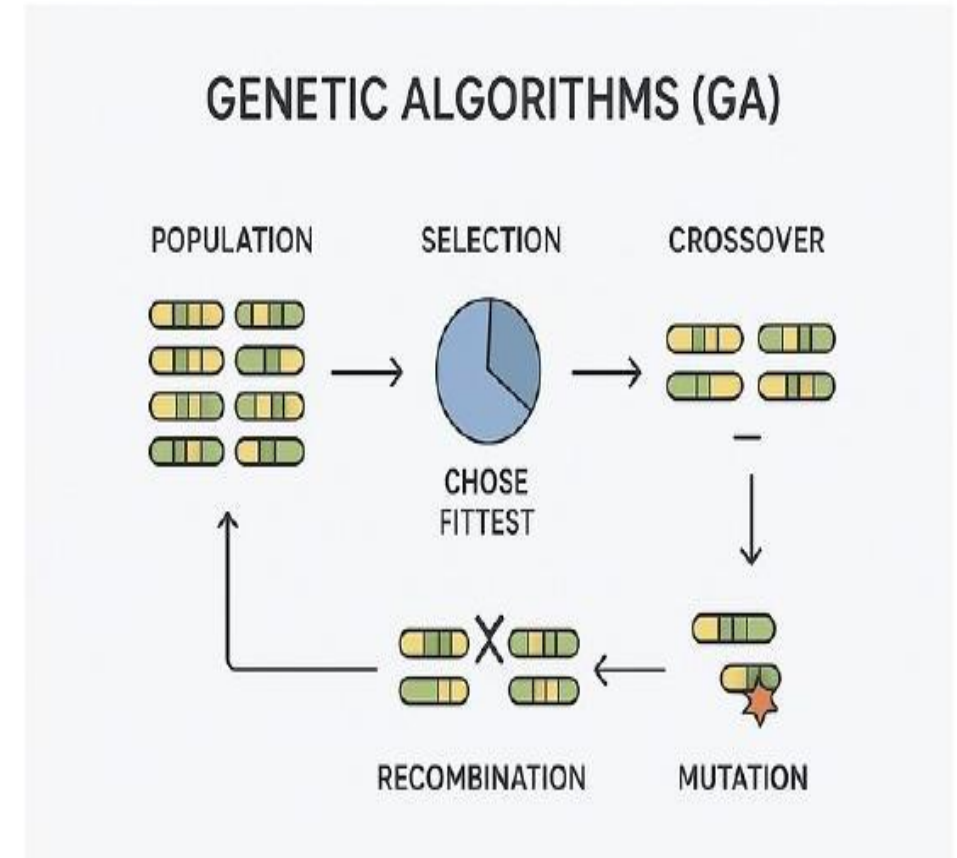


Local Beam Search

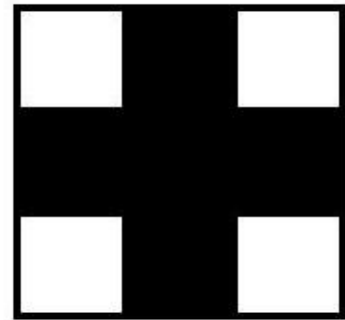
- **Keep k states instead of just one, generate all their successors, keep the best k overall**
- Not the same as k independent random restarts — the k states share information: a good state can 'recruit' the whole search toward its neighborhood
- Can suffer from lack of diversity — all k states clustering in the same region of the landscape
- **A natural stepping stone toward maintaining a whole population — which is exactly what a genetic algorithm does next**

Genetic Algorithms

- A population-based search method inspired by biological evolution
- Each state is encoded as a chromosome (often a bit string)
- **Selection** — fitter individuals are more likely to reproduce
- **Crossover** — parents exchange parts of their chromosomes
- **Mutation** — small random changes maintain diversity
- The population evolves toward better solutions over generations



Genetic Algorithm Example: A Minimal Trace



Goal: [0,1,0,1,1,1,0,1,0]

1. Initial:

[0,1,0,1,0,0,0,1,0]
 [0,1,0,0,1,0,1,1,0]
 [1,0,0,1,0,1,0,0,0]
 [0,1,0,1,0,1,0,1,1]
 [0,1,0,0,0,0,0,1,0]

2. Crossover:

[0,1,0,1,0,1,0,1,1]
 [0,1,0,0,0,0,0,1,0]



[0,1,0,0,0,0,0,1,1]
 [0,1,0,1,0,1,0,1,0]

3. Mutation:

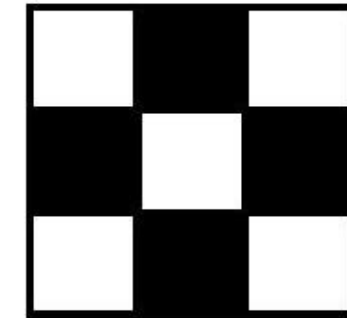
[0,1,0,1,0,1,0,1,1]
 [0,1,0,0,1,0,0,1,0]

4. Evaluation

[0,1,0,1,0,0,0,1,0]
 [0,1,0,0,1,0,1,1,0]
 [1,0,0,1,0,1,0,0,0]
 [0,1,0,1,0,1,0,1,1]
 [0,1,0,0,0,0,0,1,0]
 [0,1,0,1,0,1,0,1,1]
 [0,1,0,0,1,0,0,1,0]

5. Selection

[0,1,0,1,0,0,0,1,0]
 [0,1,0,0,1,0,1,1,0]
~~[1,0,0,1,0,1,0,0,0]~~
 [0,1,0,1,0,1,0,1,1]
~~[0,1,0,0,0,0,0,1,0]~~
 [0,1,0,1,0,1,0,1,1]
 [0,1,0,0,1,0,0,1,0]



Repeat N times.

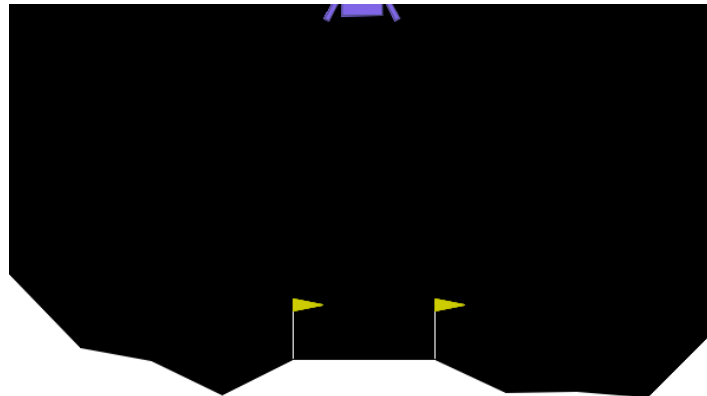
Conclusions

- Classical search: systematic exploration of paths, from blind (BFS/DFS/UCS) to informed (Greedy/A*)
- **A heuristic doesn't just speed things up — admissibility and consistency determine whether the answer is still guaranteed correct**
- Local search drops the path entirely: $O(1)$ memory, at the cost of completeness and optimality guarantees
- **Hill climbing, simulated annealing, and genetic algorithms are three different answers to the same question: how do you move through a landscape you can only see locally?**

Gymnasium



**An API standard for reinforcement learning with a
diverse collection of reference environments**



https://github.com/anton-selitskiy/SB_TAI/blob/main/Week01/search_algorithms_frozenlake.ipynb